

Temperature Prediction Model

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR

from sklearn.metrics import mean_squared_error, r2_score
```

1. Data Loading and Initial Inspection

```
# Load the dataset
df = pd.read_csv("/content/weather_description.csv")

print("Dataset Loaded")
print("Shape:", df.shape)
print("\nFirst 5 rows of the dataset:")
display(df.head())
```

2. Data Preprocessing

```
# Convert 'datetime' column to datetime objects
df['datetime'] = pd.to_datetime(df['datetime'])

# Handle missing values using forward fill then backward fill, then drop any remaining NaNs
df = df.ffill()
df = df.bfill()
df = df.dropna()

print("Missing values after cleaning:")
print(df.isnull().sum().sum())

Missing values after cleaning:
0
```

3. Feature Engineering and Target Definition

```
# Extract time-based features
df['hour'] = df['datetime'].dt.hour
df['day'] = df['datetime'].dt.day
df['month'] = df['datetime'].dt.month

# Define the target city and features
target_city = "Vancouver"
X = df[['hour', 'day', 'month']]
y = df[target_city]

print("Target missing values:", y.isnull().sum())
```

Target missing values: 0

```
from sklearn.preprocessing import LabelEncoder

# Initialize LabelEncoder
le = LabelEncoder()

# Fit and transform the target variable 'y'
y = pd.Series(le.fit_transform(y), index=y.index)

print("Categorical target variable 'Vancouver' converted to numerical labels.")
print("First 5 encoded target values:")
display(y.head())
```

4. Data Splitting and Scaling

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

print("Training Size:", X_train.shape)
print("Testing Size:", X_test.shape)
```

Training Size: (36202, 3)
Testing Size: (9051, 3)

```
# Scale the features using StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Features scaled successfully.")
```

Features scaled successfully.

5. Model Training and Evaluation

```
# Initialize models
models = {
    "Linear Regression": LinearRegression(),
```

```

"Random Forest": RandomForestRegressor(n_estimators=50, random_state=42),
"KNN": KNeighborsRegressor(),
"SVM": SVR()
}

results = {}

print("\nMODEL TRAINING AND TESTING\n")

# Train and evaluate each model
for name, model in models.items():
    print("=====")
    print("Algorithm:", name);
    print("=====")

    model.fit(X_train, y_train);
    pred = model.predict(X_test);

    mse = mean_squared_error(y_test, pred);
    r2 = r2_score(y_test, pred);

    results[name] = r2;

    print("Mean Squared Error:", mse);
    print("R2 Score:", r2);
    print("\n");

```

```

MODEL TRAINING AND TESTING

=====
Algorithm: Linear Regression
=====
Mean Squared Error: 91.86117368325534
R2 Score: 0.0026081728442591823

=====
Algorithm: Random Forest
=====
Mean Squared Error: 116.25696934362172
R2 Score: -0.26227160424753104

=====
Algorithm: KNN
=====
Mean Squared Error: 108.71671196552867
R2 Score: -0.18040251002615348

=====
Algorithm: SVM
=====
Mean Squared Error: 99.4881553078584
R2 Score: -0.08020253850671866

```

6. Model Comparison and Visualization

```

# Plot algorithm comparison
plt.figure(figsize=(8,6))
plt.bar(results.keys(), results.values())
plt.title("Algorithm Comparison")
plt.ylabel("R2 Score")
plt.show()

```

```
# Identify and print the best performing algorithm
best_algorithm = max(results, key=results.get)
print("\nBEST ALGORITHM IS:", best_algorithm)
print("WITH R2 SCORE:", results[best_algorithm])
```

```
BEST ALGORITHM IS: Linear Regression
WITH R2 SCORE: 0.0026081728442591823
```

7. Actual vs. Predicted Temperature for Best Model

```
# Retrain the best model for detailed analysis if needed (or use the one from results)
best_model = models[best_algorithm] # Get the instance of the best model
best_model.fit(X_train, y_train) # Ensure it's fitted if not already
pred = best_model.predict(X_test)

# Plot Actual vs Predicted temperatures
plt.figure(figsize=(6,6))
plt.scatter(y_test, pred)
plt.xlabel("Actual Temperature")
plt.ylabel("Predicted Temperature")
plt.title(f"Actual vs Predicted Temperature ({best_algorithm})")
plt.show()
```

8. Temperature Trend

```
# Plot the trend for the target city's encoded weather descriptions
plt.figure(figsize=(10,5))
```

```
plt.plot(df[target_city][:500]) # Plotting first 500 data points for clarity
plt.title(f"Encoded Weather Description Trend for {target_city}")
plt.xlabel("Data Point Index")
plt.ylabel("Encoded Weather Description")
plt.show()
```

```
plt.figure(figsize=(8,6))

plt.bar(results.keys(), results.values())

plt.title("Algorithm Comparison")

plt.ylabel("R2 Score")

plt.show()
```

```
best_model = RandomForestRegressor()

best_model.fit(X_train, y_train)

pred = best_model.predict(X_test)

plt.figure(figsize=(6,6))

plt.scatter(y_test, pred)

plt.xlabel("Actual Temperature")

plt.ylabel("Predicted Temperature")
```

```
plt.title("Actual vs Predicted")  
  
plt.show()
```

```
plt.figure(figsize=(10,5))  
  
plt.plot(df[target_city][:500])  
  
plt.title("Temperature Trend")  
  
plt.show()
```

```
best_algorithm = max(results, key=results.get)  
  
print("\nBEST ALGORITHM IS:", best_algorithm)  
  
print("WITH R2 SCORE:", results[best_algorithm])
```

```
BEST ALGORITHM IS: SVM  
WITH R2 SCORE: 0.7874308162891662
```